

Data Agent 前端设计与架构文档

基于传统 BI 设计原则、ThoughtSpot 公开资料反推，以及 “Analysis Story / Insight Card / Canvas / Dashboard” 产品抽象整理

版本 1.0 · 适用对象：产品、前端、设计、数据平台、Agent 应用架构 · 产出目标：形成可实现、可演进、可治理的前端蓝图

执行摘要

- 本产品不应做成“聊天页 + 若干图表”的组合，而应做成以 Analysis Story 为核心对象的分析工作区。每次 AI 回答都应是一个可操作的微型 dashboard / analysis story，而不是一条孤立 message。
- 传统 BI 的强项不是图表本身，而是稳定、可信、可复用、可治理的分析资产；AI 的价值不是替代这套骨架，而是降低发现、下钻、解释、验证与沉淀的摩擦。
- 前端应采用 Host Shell、AI Surface、Story / Canvas、Application State、Event Bus、Runtime Boundary 的分层架构。这样可以同时支持全局 AI、对象级 AI、可视化分析、语义对象编排和后端多步分析执行。
- 页面骨架建议采用“中间常驻、左右按需展开”的三列式壳布局：左边解决“去哪儿”，中间承载“看什么、问什么、继续怎么走”，右边承载“为什么可信、还有什么细节”；移动端退化为单列主视图 + 半覆盖 Inspect。
- 页面的中心应从 message list 迁移为 StoryCard list。Agent 先做一轮可见分析，用户可以接管、修正、分叉、保存与分享；系统的学习闭环来自这些中间动作，而不只是最终结论。

一句话总纲： BI 负责稳定表达真相，Agent 负责降低获取真相的摩擦；最终产品形态应是“BI 的骨架，Agent 的交互”。

1. 文档目标与边界

本文不是某个具体前端框架的 API 手册，而是一份面向自研 Data Agent 产品的前端设计与架构说明。它覆盖如下内容：

- 产品抽象：如何定义 Analysis Story、Insight Card、Canvas、Dashboard 之间的关系。
- 信息架构：如何把全局 AI、对象级 AI、分析结果、下钻控制和资产沉淀放在同一页面中。
- 前端架构：如何划分壳层、运行时边界、事件总线、状态域和组件职责。

- 实现路线：如何从最小可用形态逐步演进到稳定的多对象分析工作区。

不在本文范围内的内容包括：具体图表库选型、具体 UI 样式 token、后端 Agent 算法细节、SQL 编排细节以及组织内权限模型实现细则。

2. 设计出发点：从传统 BI 学什么，从 AI 时代改什么

传统 BI 的设计哲学提供了极强的基础骨架，尤其体现在以下几个方面：

- 先保证共识，再追求智能：统一指标口径、固定维度、稳定时间粒度、可复现结果。
- 页面首先服务“扫一眼就懂”：顶部 KPI、中部趋势与对比、底部明细与解释。
- 过滤器与下钻路径是分析语言的一部分，不应被纯对话完全替代。
- Dashboard 是长期资产，而不是一次性回答；保存、订阅、分享、告警都必须是第一公民。
- 性能、权限、治理、审计不是后补事项，而是产品可信度的一部分。

AI 时代需要改造的不是这些原则，而是其交互摩擦。传统 BI 把“观察、分析、下钻、推理、假设、验证、转向”留给用户手工完成；Agent 则应该自动执行其中的一部分，并将过程显化给用户，使用户能够接管与反馈。

维度	传统 BI 强项	AI 时代需要增强	本文设计结论
结果表达	结构稳定、易复用	更低摩擦、更强解释	保留 dashboard 资产逻辑，增加 Analysis Story
探索路径	下钻稳定、可审计	可动态分叉、可追问	Agent 与用户共用同一套分析动作语义
资产沉淀	保存/分享/订阅	从即时答案自然沉淀	StoryCard 可 pin 到 Canvas，再保存为 Dashboard
治理可信	权限、口径、审计	Agent 不应绕开治理	前端通过 Runtime Boundary 连接语义层与权限边界

3. 核心产品判断

1. 每次 AI 回答都应该是一个可操作的微型 dashboard / analysis story，而不只是文本回复。
2. Canvas 不是单纯摆卡片的地方，而是多个分析故事并存、对比、编排和沉淀的工作区。

3. Dashboard 不是另一个完全不同的实体；它更像是被保存、布局稳定、可复用的 Canvas 结果。
4. Agent 的价值不只是“会说结论”，而是“会分析、会展示分析过程、也允许人接管分析过程”。
5. 页面中心不应是消息流，而应是 StoryCard 列表与当前工作区。

分析对象模型：从 AI 回答到 Dashboard 资产

核心抽象：Answer 不是 message，而是可操作的微型 dashboard / analysis story。



一个 Story Card 的最小结构



设计原则：Agent 先做一轮可见分析，用户可以接管、修正、分叉与沉淀；系统学习来自这些中间动作，而不只是最终答案。

图 1. 分析对象模型：从问题到 Story，再到 Canvas 与 Dashboard。

4. 术语与对象模型

术语	定义	为什么重要
Analysis Story	一次分析场景的完整对象，包含 question、conclusion、evidence、trace、next moves 与 context。	它是前端的最小一等公民。
Insight Card / StoryCard	Analysis Story 在画布中的具体 UI 表现。	页面不再以 message 为中心，而以卡片化分析对象为中心。
Evidence Block	KPI、图表、表格、叙述块等证据单元。	支持 Story 内部的多模态呈现。
Trace	Agent 已经执行过的分析动作列表。	让用户看见过程，建立信任并便于纠偏。
Next Moves	系统建议用户继续执行的动作。	把对话从“继续问”升级为“继续分析”。
Canvas	一个能容纳多个 Story 的编排工作区。	支持横向对比、分支分析和沉淀。
Dashboard	被保存的、布局稳定的 Canvas 结果。	承接传统 BI 的资产属性。

对象化原则： 在前端内部，Story、Chart、Table、Dashboard 都应当是可寻址、可持久化、可上下文追问的对象；不要把它们仅仅当作一次渲染结果。

5. 用户体验模型：对象级 AI + 全局 AI 双层入口

一个成熟的数据代理页面不应只有一个聊天框。更合理的方式是同时提供两类 AI 入口：

- Global AI：位于顶栏，用于跨页面、跨对象、跨指标的全局提问与发起新分析。
- Contextual AI：附着在 KPI、图表、表格、标题等对象上，用于“解释这个对象”“基于当前对象追问”“把当前对象转成动作”。

对象级 AI 的默认动作不应是“等待用户输入”，而应是“先给一个默认解读，再允许用户继续追问或接管”。这会显著降低使用门槛，并让 AI 成为页面级能力而非独立页面。

页面信息架构：对象级 AI + 全局 AI 双层入口

分析页不是单一聊天页；AI 是页面级能力，既可以全局提问，也可以对具体对象进行解释、追问与操作。

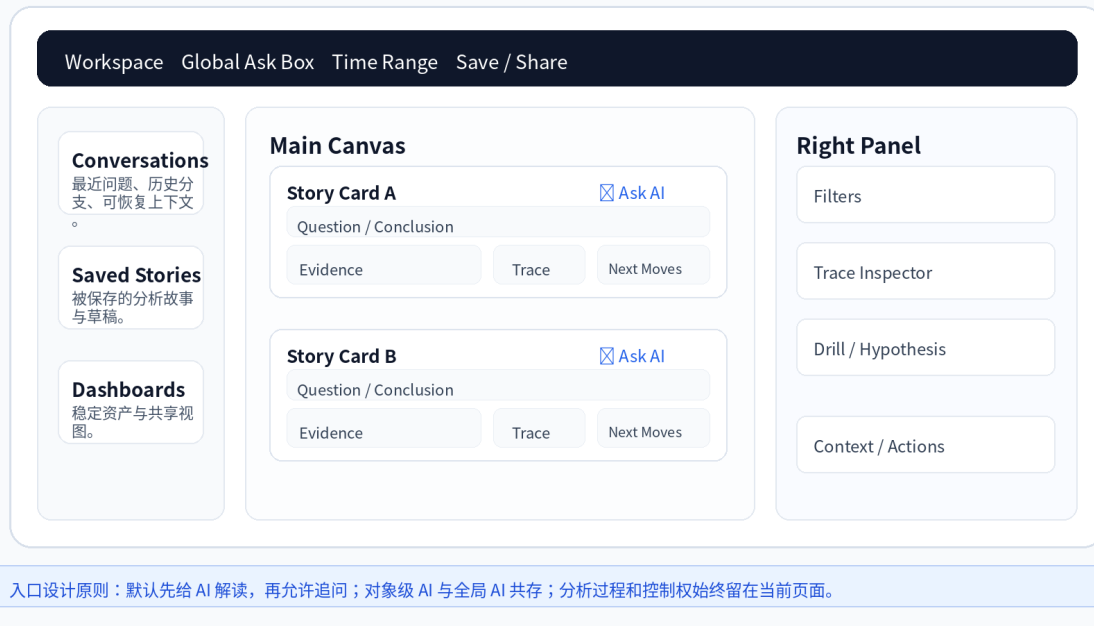


图 2. 页面信息架构：全局 AI 与对象级 AI 共存，右侧面板负责对当前 Story 的接管。

6. 页面结构与区域职责

区域	主要职责	不应承担的职责
Top Bar	Workspace 切换、Global Ask、时间范围、保存与分享。	不直接承载复杂分析结果。
Left Rail	会话历史、已保存 Story、Dashboards。	不承载复杂配置逻辑。
Main Canvas	StoryCard 列表、证据块、内联追问、卡片编排。	不直接管理底层查询执行。
Right Panel	Filters、Trace Inspector、Drill / Hypothesis、Context / Actions。	不成为第二个聊天流。

6.1 三列式壳布局原则

页面骨架建议参考 ChatGPT 式三列壳：中间 Conversation / Story Canvas 始终是主工作面，左右两侧按需展开。这样既能保留对话与分析的连续性，又能把导航、证据、元数据和工件细节从主画布中解耦出来。

这个布局的核心不是“做成三栏”本身，而是让三块区域各自承担稳定职责，并在桌面端与移动端之间保持一致的心智模型。

区域	主责	展开策略	典型内容
Left Nav	解决“去哪儿”	桌面端可折叠；移动端沉到底部导航或抽屉	New chat、Dashboards、Threads、Memory、Projects
Center Conversation	解决“看什么、问什么、继续怎么走”	始终保持可用，是页面主工作面	User / Agent turns、Insight card、图表卡、Key findings、follow-up 输入框
Right Inspect	解决“为什么可信、还有什么细节”	默认按需展开，可关闭；移动端退化为半覆盖 Inspect sheet	Process、Evidence、Metadata、Artifacts、Citations

表 1. 三列式壳布局的职责分工。

6.2 桌面端：中间常驻，左右按需展开

桌面端默认形态建议采用左导航、中画布、右 Inspect 的三列式布局：

- Left Nav：容纳 New chat、Dashboards、Threads、Memory、Projects 等“导航型对象”。
- Center Conversation：承载用户问题、Agent 过程提示、Insight card、图表卡、关键发现与 follow-up 输入框，是整个系统的常驻主区域。
- Right Inspect：用标签页容纳 Process、Evidence、Metadata、Artifacts 等检查型信息；这些信息很重要，但不应持续挤占主画布。
- 行为原则：Left / Right 都允许折叠；Center 必须始终可用。页面在任何时刻都不能因为侧栏关闭而失去主要分析能力。

6.3 折叠态桌面端：中心体验优先于侧边信息

当屏幕宽度有限、用户进入专注分析状态、或左/右侧栏被手动关闭时，布局应退化为“中间单列为主，侧边能力通过顶部入口再打开”。此时页面不应看起来像“功能消失”，而应表现为“详细面板暂时收起”。

状态	保留内容	入口变化
默认桌面端	左导航 + 中画布 + 右 Inspect	左右两侧支持显式折叠
左侧折叠	中画布 + 右 Inspect	导航入口收进顶栏或抽屉按钮

右侧折叠	左导航 + 中画布	Inspect 通过按钮、快捷键或对象级入口打开
------	-----------	--------------------------

表 2. 桌面端不同折叠状态下的体验保持策略。

6.4 移动端：单列主视图 + 半覆盖 Inspect

移动端不适合强行保留三列，但仍应保持同一套职责分配。最稳妥的做法是：主屏只保留 Conversation / Story Canvas；导航沉到底部栏或抽屉；Inspect 通过半覆盖 sheet 打开。

- 主视图：显示用户问题、Agent 过程提示、Insight card、图表预览、关键发现与 follow-up 输入框。
- 底部导航：承载 Home、Thread、Inspect、More 等高频页面级入口。
- Inspect 半覆盖：用于查看 Process、Evidence、Metadata、Artifacts，避免打断主画布上下文。
- 交互原则：任何需要“看更多细节”的动作都进入 Inspect；任何需要“继续分析”的动作都优先留在主视图。

一句话总结：左边解决“去哪儿”，中间解决“看什么、问什么、继续怎么走”，右边解决“为什么可信、还有什么细节”。

在这个壳布局之上，StoryCard 才成为真正的中心对象：它既能承接 Conversation，又能承接证据、动作与沉淀逻辑。

7. StoryCard 的内部结构

每个 StoryCard 都应有一套固定的层次，以确保不同回答之间的结构一致、可比较、可复用：

- Question：此次分析究竟在回答什么问题。
- Conclusion：一句话结论或当前最佳判断。
- Evidence：支撑结论的 KPI、图表、表格和文字说明。
- Trace：Agent 已执行的动作链路，例如“看总体 → 对比去年 → 按渠道拆分 → 下钻到城市”。
- Next Moves：供用户点击的后续分析动作，例如“看门店层”“换利润指标”“对比华南”。

设计建议： StoryCard 的视觉重心应始终放在 Conclusion 与 Evidence 上；Trace 应可见但不喧宾夺主；Next Moves 应足够少且足够明确。

8. 前端分层架构

为了避免页面、状态、AI 和分析执行逻辑强耦合，建议把前端拆成以下分层：

前端分层架构：Host Shell + Analysis Runtime + Event Bus

目标：把页面壳、对象模型、AI 入口、分析执行边界解耦，避免做成一个巨大的“聊天 + 图表”页面。



图 3. 推荐的前端分层：Host Shell、AI Surface、Story / Canvas、State、Event Bus、Runtime Boundary。

8.1 App Shell

App Shell 负责路由、认证、工作区切换、主题、权限边界和导航。它需要稳定、薄、可长期演进，但不应直接承担具体分析执行。

8.2 AI Surface

AI Surface 负责全局提问框、对象级 Ask AI 按钮、Insight Panel 以及由系统自动生成的追问和建议动作。它是用户与分析系统交互的入口层。

8.3 Story / Canvas Layer

Story / Canvas Layer 是产品价值最集中的地方：它让回答成为对象，让多个对象形成工作区，并支持沉淀为 Dashboard。

8.4 Application State

建议把工作区状态、会话状态和 UI 状态拆开，避免把所有信息都塞进一个 message list。

8.5 Event Bus / Controller

事件总线负责解耦组件和运行时。UI 组件不直接彼此驱动，而通过标准化事件进入 Controller，再由 Controller 调用后端 Agent / Data Kernel。

8.6 Runtime Boundary

Runtime Boundary 是前端与语义层、查询执行、Agent orchestration 之间的接口边界。前端应发送分析意图和上下文，而不是直接在组件内部编排 SQL。

9. 前端对象模型与数据结构

推荐把 Analysis Story 作为前端的最小业务对象，并围绕它定义证据块、动作、上下文等结构。以下类型足以支撑第一版实现：

```
type AnalysisStory = {
  id: string
  question: string
  status: "planning" | "running" | "done" | "error"

  summary?: string
  conclusion?: string

  evidence: EvidenceBlock[]
  trace: AnalysisStep[]
  nextMoves: NextMove[]

  context: AnalysisContext
  layout: StoryLayout
  persistedAs?: { type: "dashboard" | "saved_answer"; id: string }
}

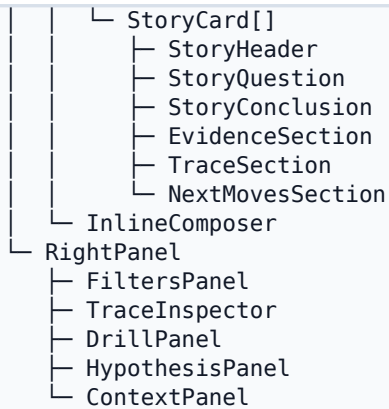
type AnalysisContext = {
  assetId?: string
  timeRange?: TimeRange
  filters: Filter[]
  metrics: string[]
  dimensions: string[]
  selection?: SelectionState
}
```

一个重要原则是：上下文必须显式存在，而不是只隐式存在于对话历史中。只有这样，系统才能稳定地继承当前卡片的过滤器、粒度、选中项与语义模型。

10. 组件树与职责边界

推荐的 React 组件层次如下：

```
AppShell
├── WorkspacePage
│   ├── TopBar
│   │   ├── WorkspaceSwitcher
│   │   ├── GlobalAskBox
│   │   ├── TimeRangePicker
│   │   ├── SaveButton
│   │   └── ShareButton
│   ├── LeftRail
│   │   ├── ConversationList
│   │   ├── SavedStoriesList
│   │   └── DashboardList
│   ├── MainCanvas
│   └── StoryGrid
```



关键边界是：StoryCard 负责展示故事，不负责调度 Agent；RightPanel 负责接管当前 Story，不直接重写整页；Controller 负责把用户动作与 Agent 执行动作统一到同一套分析语义中。

11. 状态管理建议

状态域	包含内容	为什么单独拆分
WorkspaceStore	stories、storyOrder、activeStoryId、canvas 布局。	这是页面的主业务状态。
ConversationStore	用户输入、系统消息、历史分支。	避免把业务对象与聊天历史混在一起。
UIStore	右侧标签、弹层开关、当前选中证据块。	UI 噪声不污染业务对象。

```

type WorkspaceStore = {
  activeStoryId?: string
  storyOrder: string[]
  stories: Record<string, StoryCardVM>
}

type ConversationStore = {
  messages: ChatMessage[]
}

type UIStore = {
  rightPanelTab: "filters" | "trace" | "drill" | "hypothesis" | "context"
  selectedEvidenceId?: string
}
  
```

12. 事件总线与动作语义

一个优秀的数据代理前端，不应让组件之间通过 props 链条互相驱动。更好的做法是定义统一事件总线。事件总线的目标有两个：一是解耦；二是让 Agent 与用户共享同一套动作语义。

```

type AnalysisAction =
  | { type: "OBSERVE_TREND" }
  | { type: "BREAKDOWN"; by: string }
  | { type: "DRILL_DOWN"; by: string; value?: string }
  | { type: "COMPARE"; against: string }
  | { type: "FORM_HYPOTHESIS"; text: string }
  | { type: "VALIDATE_HYPOTHESIS"; text: string }
  | { type: "PIVOT"; reason: string }
  | { type: "CHANGE_METRIC"; metric: string }
  | { type: "APPLY_FILTER"; filters: Filter[] }

```

当 Agent 执行这些动作时，它们会变成 Trace；当用户点击“继续下钻”“改成利润指标”“对比华南”时，本质上也是触发同样的动作。这样，系统才真正实现了“Agent 先做，人再接管”。

13. 典型交互流

1. 用户在 Global Ask Box 提交问题。
2. 前端立即创建一个空的 StoryCard，并进入 planning / running 状态。
3. Controller 把问题与当前上下文发给后端 Agent / Data Kernel。
4. 系统流式返回 Trace、Evidence、Conclusion 和 Next Moves，前端逐步填充 StoryCard。
5. 用户可点击某个 Next Move、修改过滤器或在对象级 AI 入口继续追问；系统可以更新当前 Story，也可以基于当前 Story 分叉出新 Story。
6. 用户将一个或多个 Story pin 到 Canvas，并最终保存为 Dashboard。

流式策略建议： 优先返回问题标准化结果与第一条 Trace，再返回第一批 Evidence，随后更新 Conclusion，最后给出 Next Moves。这样用户会感到系统在真实思考，而不是长时间空白等待。

14. 前后端接口边界

前端应传递分析意图与上下文，而不是把底层查询编排写死在页面中。一个简洁而稳妥的接口如下：

```

type RunAnalysisRequest = {
  question: string
  context?: AnalysisContext
  allowedActions?: AnalysisAction["type"][]
}

type RunAnalysisResponse = {
  story: AnalysisStory
}

```

这样做的好处是：未来无论后端使用单 Agent、多 Agent、semantic query、lineage 检索还是规则引擎，前端接口都可以保持稳定。

15. 主题定制、权限与治理

前端设计不能只考虑视觉，还必须在 Day 1 就为嵌入式品牌定制、权限边界和治理能力留出接口。

- 主题定制：颜色 token、圆角、密度、logo、字体尺度、图标风格不应写死。
- 权限边界：前端必须明确知道哪些 Story、过滤器、下钻路径和对象级动作可见或不可见。
- 治理透明：StoryCard 应能挂接 lineage、数据来源、更新时间、适用权限和指标说明。
- 审计与观测：记录用户触发了哪些动作、接受了哪些建议、在哪一步放弃，从而反向优化 Agent 行为。

16. 性能与体验策略

问题	推荐策略
首屏加载过重	Shell 与 Runtime 分层；首页先加载工作区壳与最近资产，分析运行时按需懒加载。
StoryCard 太多导致页面变慢	主画布虚拟化；EvidenceBlock 延迟渲染；图表进入视口后再挂载。
Agent 响应慢带来的不确定感	流式返回 Trace 与第一批证据；在 StoryCard 内显示阶段性状态。
同一问题重复计算	对标准化问题 + 上下文进行结果缓存；支持从 saved story 直接恢复。
卡片布局混乱	Canvas 保持固定栅格与有限模板，不允许无限自由拖拽破坏信息密度。

17. 反模式与风险提示

- 不要把所有东西都塞进 message list；这样会让分析对象、上下文和资产沉淀无法成型。
- 不要让前端组件直接拼查询；这会让产品极难治理，也会使 AI 与语义层脱节。
- 不要把 Trace 隐藏成黑盒；如果用户看不见系统做过什么动作，就无法建立信任，也无法有效反馈。
- 不要让 Next Moves 太多；系统给出的建议动作必须像分析助手而不是菜单目录。
- 不要过早追求无限自由布局；在产品早期，信息层级和交互确定性比花哨的排版自由更重要。

18. 分阶段落地路线

阶段	目标	核心交付
Phase 1	做出最小可用的分析工作区	App Shell、Global Ask、单 StoryCard、Conclusion + Evidence + Trace + Next Moves、右侧过滤器面板。
Phase 2	让回答变成可复用对象	Saved Story、Pin to Canvas、Dashboard 保存、对象级 Ask AI、事件总线标准化。
Phase 3	形成多对象分析系统	多 Story 对比、分支分析、共享语义动作、Dashboard Composer、主题与权限扩展。
Phase 4	形成闭环优化能力	用户动作观测、Next Move 优化、模板化分析路径、资产推荐与治理增强。

19. 基于公开资料的 ThoughtSpot 反推依据

以下公开信息对本设计有直接启发。这里不试图猜测其内部实现细节，而是提炼对自研架构有价值的外部证据：

公开证据	可观察到的事实	对本文设计的启发
React Embed 文档	ThoughtSpot 公开提供 App / Liveboard / Search / Spotter 等可嵌入对象，并通过 HostEvent / EmbedEvent 与宿主交互。	说明产品是对象型 surface，而不是单一聊天页。
Liveboard 文档	Liveboard 被描述为由用户 pin 上去的一组可视化组成的交互式 dashboard。	印证 “Story / Card 可以沉淀为 dashboard 资产”。
Spotter / Spotter Agent 文档	AI analyst 能以嵌入对象形式存在，支持更深度集成。	说明 AI 入口可以是页面级能力，而不必是独立页面。
Visual Embed SDK 文档	支持搜索、图表、表格、自动化	说明壳层、运行时边界与事件总

	insight 与 full app 嵌入。	线是值得借鉴的架构方向。
--	------------------------	--------------

参考资料（官方文档标题）

- Embed with React components — ThoughtSpot Developers
- Embed a Liveboard — ThoughtSpot Developers
- Basic Liveboard usage — ThoughtSpot Cloud
- Embed Spotter experience — ThoughtSpot Developers
- HostEvent / EmbedEvent — ThoughtSpot Developers
- ThoughtSpot Embedded Documentation — ThoughtSpot Developers

20. 附录：建议的最小实现骨架

```

type AnalysisEvent =
  | { type: "story.created"; storyId: string }
  | { type: "trace.appended"; storyId: string; step: AnalysisStepVM }
  | { type: "evidence.appended"; storyId: string; block: EvidenceBlockVM }
  | { type: "conclusion.updated"; storyId: string; conclusion: string }
  | { type: "nextMoves.updated"; storyId: string; moves: NextMoveVM[] }

type StoryCardVM = {
  id: string
  question: string
  status: "planning" | "running" | "done" | "error"
  conclusion?: string
  evidence: EvidenceBlockVM[]
  trace: AnalysisStepVM[]
  nextMoves: NextMoveVM[]
  context: AnalysisContext
}

```

建议先围绕 StoryCard、WorkspaceStore、AnalysisEvent 三件事稳定下来，再去补充复杂布局、更多图表类型和更丰富的多人协作能力。